

Memoria

En Java, **siempre** se pasa por **valor** (pass-by-value) ... ?

```
public static void sumar(Integer numero) {
    numero += 10;
}
```

```
Integer numero1 = 10;
sumar(numero1);
System.out.println(numero1);
```

↓
10

```
public static void sumar(Integer[] arreglo) {
    for (int i = 0; i < arreglo.length; i++) {
        arreglo[i] += 10;
    }
}
```

```
Integer[] arreglo1 = {10, 11, 12, 13};
sumar(arreglo1);
System.out.println(Arrays.toString(arreglo1));
```

↓
[20, 21, 22, 23]

```
public static void sumar(Contacto contacto) {
    contacto.cambiarNumero(contacto.obtenerNumero() + 10);
}
```

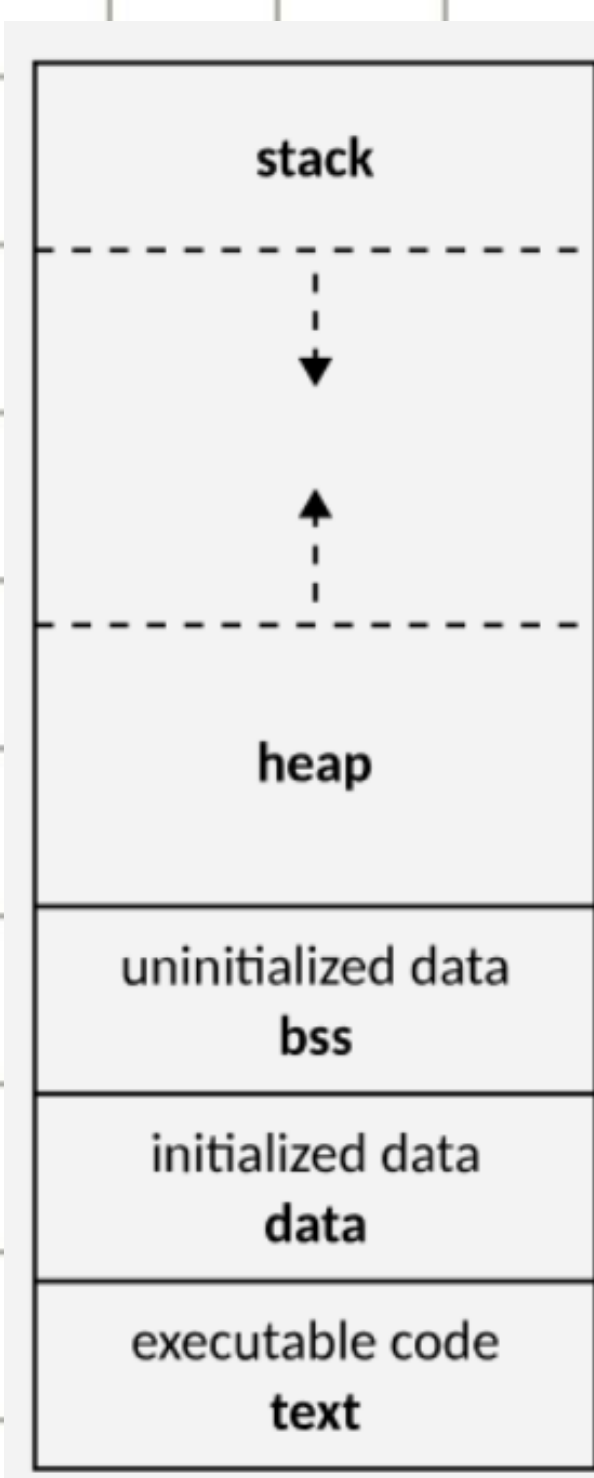
```
Contacto contacto = new Contacto(1141424344);
sumar(contacto);
System.out.println(contacto.obtenerNumero());
```

↓
1141424354

```
public static void sumarV2(Integer[] arreglo) {
    for (Integer numero : arreglo) {
        numero += 10;
    }
}
```

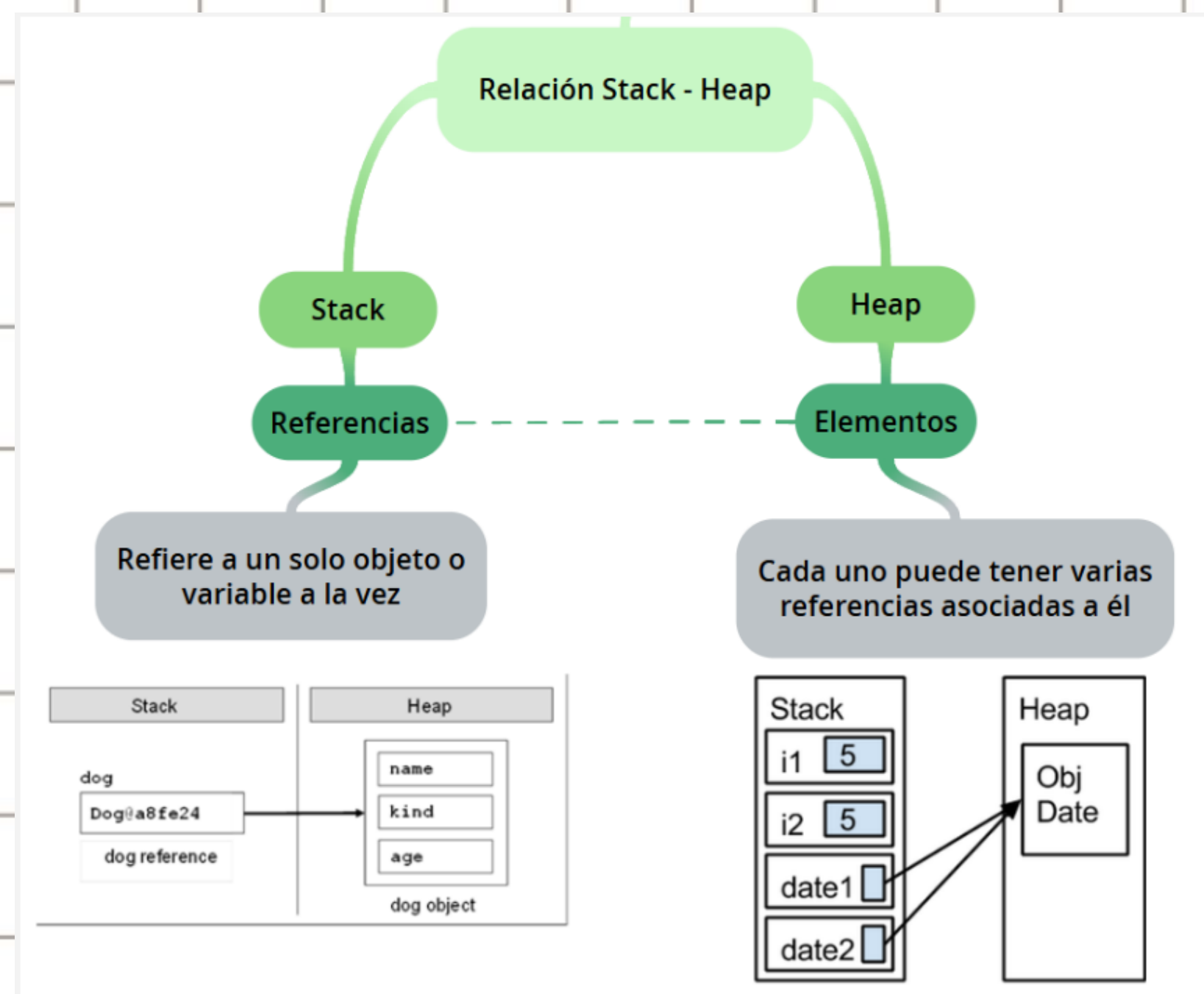
```
Integer[] arreglo2 = {10, 11, 12, 13};
sumarV2(arreglo2);
System.out.println(Arrays.toString(arreglo2));
```

↓
[10, 11, 12, 13]

Estructura de la memoria

1. **Stack**: Segmento donde viven las variables locales (se apilan).
2. **Heap**: segmento donde vive la memoria reservada dinámicamente por el Programador.
3. **Data**: segmento donde viven las variables globales y las variables locales estáticas.
4. **Code**: segmento donde vive el código máquina y las constantes.

En Java, es fundamental diferenciar las **variables** de los **objetos**. Las **variables** viven en el **stack** y guardan la **referencia** (la dirección de memoria) de un objeto. Los **objetos** viven en el **heap**, y se reservan cuando utilizamos **new**.



Se pasó por copia una variable que guardaba una referencia. Es decir, esta variable contacto apunta al mismo objeto

```
public static void sumar(Contacto contacto) {  
    contacto.cambiarNumero(contacto.obtenerNumero() + 10);  
}
```

Stack Heap

```
Contacto contacto = new Contacto(1141424344);  
sumar(contacto);  
System.out.println(contacto.obtenerNumero());
```

Se pasa por copia la variable del stack, que guarda una referencia al objeto Contacto

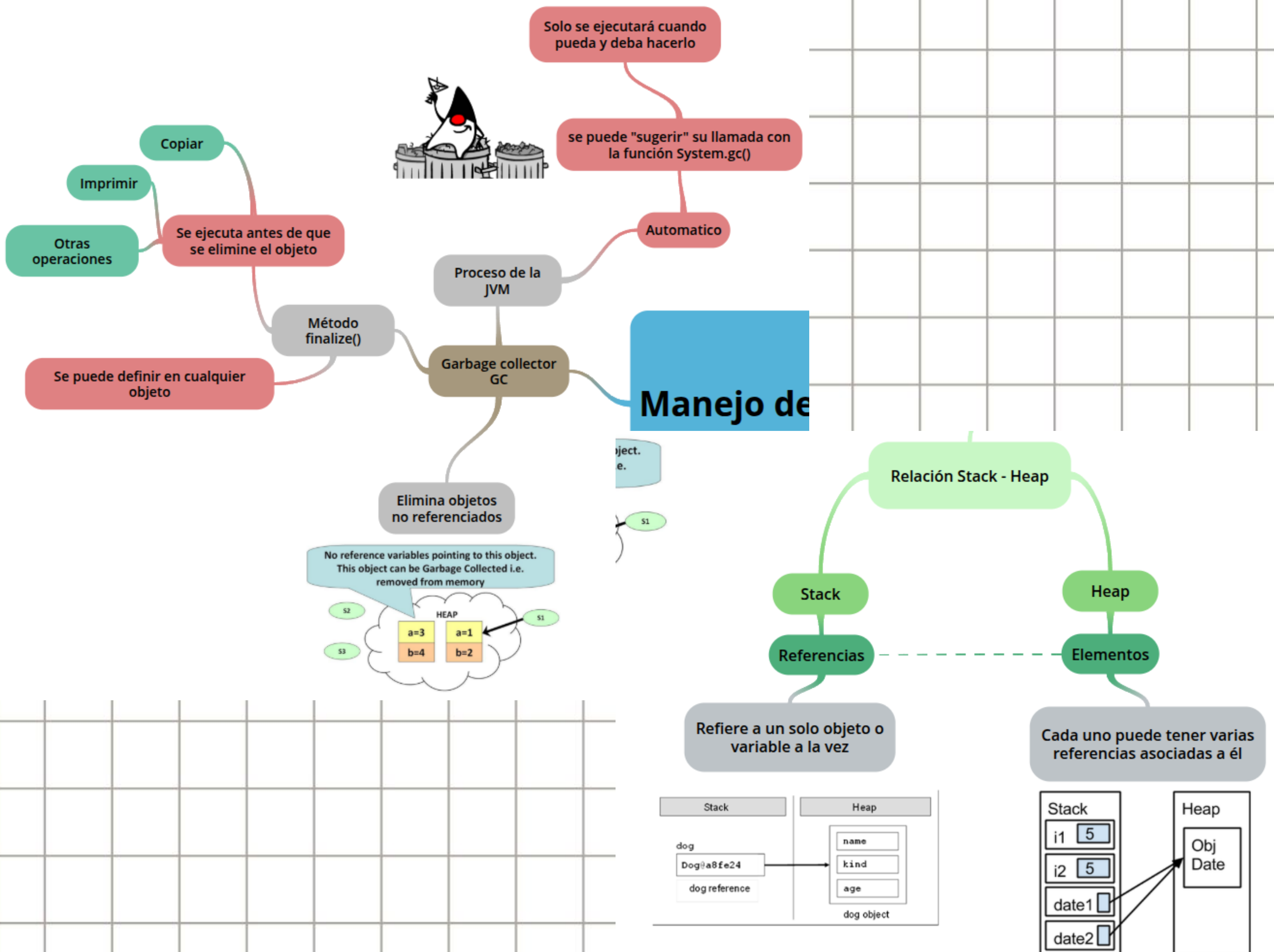
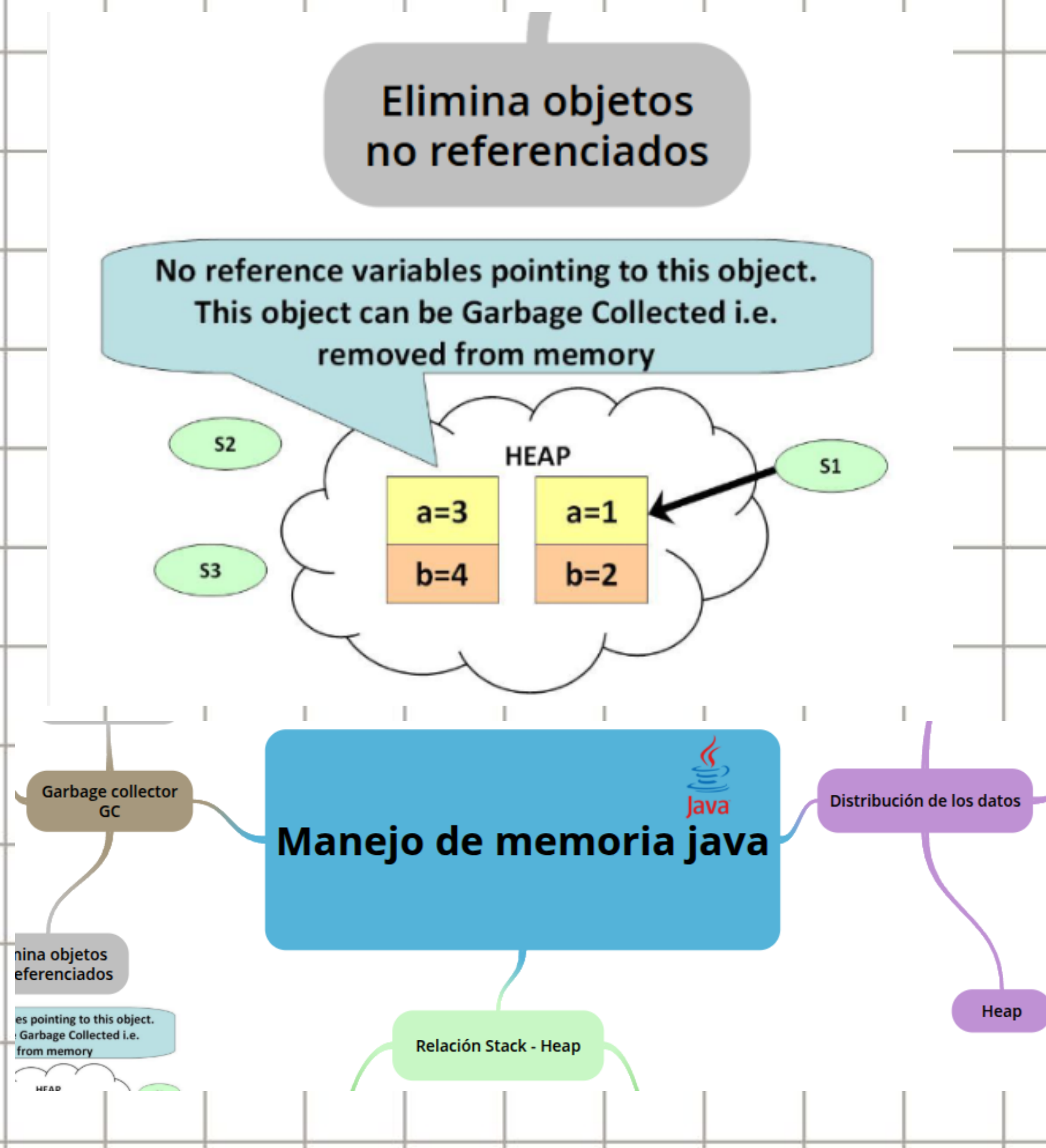
Imprime 1141424354 porque sumar() modificó el objeto al que hace referencia contacto

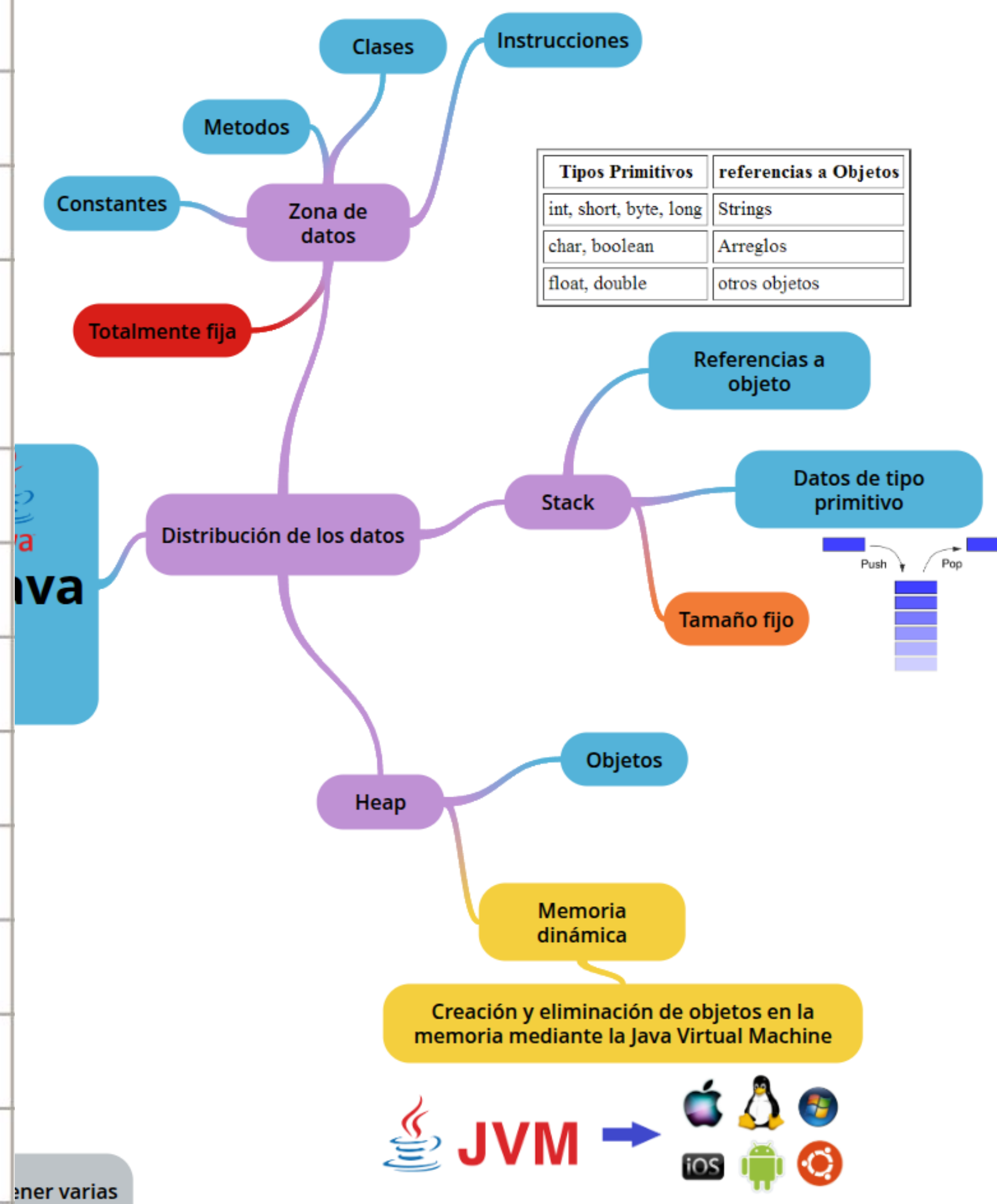
Garbage Collector

El recolector de basura (Garbage collector, GC) es una **característica de Java** que **gestiona la asignación y liberación de memoria dinámica del heap**.

Cuando un objeto **deja de tener variables haciendo referencia a él**, el GC lo **libera**.

Aunque no es mandatorio, uno puede ayudar al GC asignando null a una referencia que quiera liberar.





Ejemplo: El Señor de los Anillos



Ante la ardua tarea de llevar el Anillo al Monte del Destino, Frodo y sus compañeros arman un equipo...

```
Equipo equipo = new Equipo(3);
Miembro miembro;
Miembro frodo = new Miembro("Frodo", "Hobbit");
miembro = frodo;
equipo.agregar(miembro, 0);
Miembro gandalf = new Miembro("Gandalf", "Maiar");
miembro = gandalf;
equipo.agregar(miembro, 1);
Miembro sam = new Miembro("Sam", "Hobbit");
miembro = sam;
equipo.agregar(miembro, 2);

System.out.println(equipo);
```

